Hybrid Graph-ML Fraud Detection System on the Elliptic Bitcoin Dataset

Architecture, Implementation, and Performance Analysis

Author: Md Ahasan Kabir
Affiliation: Adelaide University
Date: April 2026
Dataset: Elliptic Bitcoin Transaction Dataset (Weber et al., 2019)

Table of Contents

1. **System Architecture Flowchart**
2. **Comprehensive Model Description**
3. **Experimental Setup**
4. **Results and Performance Figures**
    - **4.1 Complete Metrics Table**
    - **4.2 Architecture Performance Dashboard**
    - **4.3 ROC Curves**
    - **4.4 Precision-Recall Curves**
    - **4.5 Confusion Matrices**
    - **4.6 SHAP Feature Importance**
5. **SVM — In-Depth Analysis of the Best Performing Model**
    - **5.1 SVM Comprehensive Evaluation Dashboard**
    - **5.2 SVM Confusion Matrix vs Hybrid**
    - **5.3 SVM Threshold Optimisation**
    - **5.4 SVM Probability Distribution**
6. **Critical Result Analysis**
7. **Directions for Performance Improvement**
8. **References**

1. System Architecture Flowchart

The figure below presents the complete end-to-end pipeline I developed for this study. The system consists of 12 discrete, checkpointed steps that transform raw blockchain transaction data into a hybrid ensemble fraud classifier.

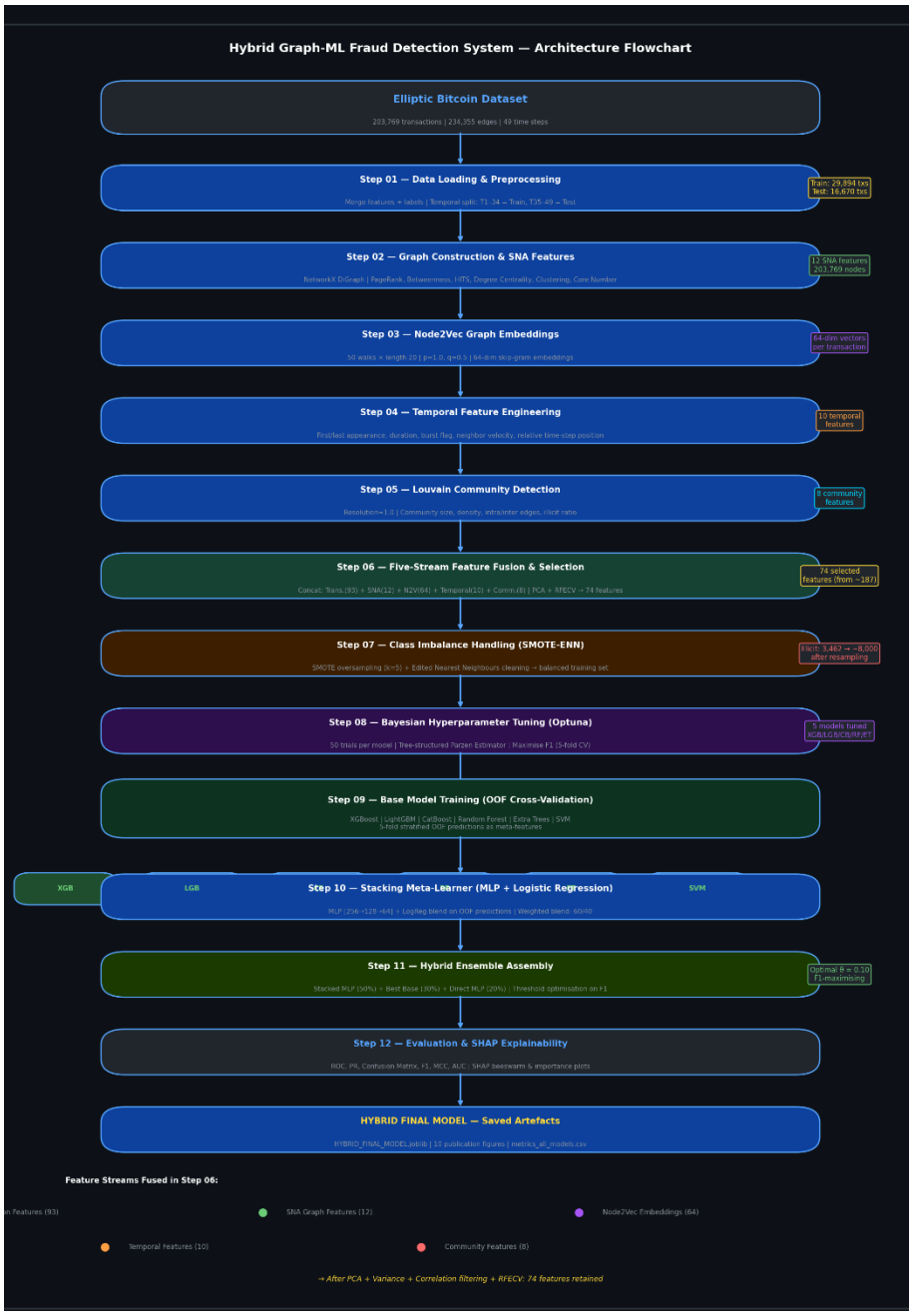

Figure 1. End-to-end pipeline architecture 12 modular steps from raw data to hybrid ensemble classifier. Each coloured block represents a module saved as a checkpoint. Side annotations show the data dimensions produced at each step.

Figure 1 shows end-to-end hybrid fraud detection pipeline. Data flows from the raw Elliptic Bitcoin Dataset (top) through feature engineering, graph analysis, resampling, and ensemble training, before being assembled into the Hybrid Final Model (bottom). Each step saves a checkpoint to results/checkpoints/, allowing re-runs to skip completed steps automatically. Side callout boxes show the data cardinality produced at each stage.

2. Comprehensive Model Description

2.1 Dataset Overview

I use the **Elliptic Bitcoin Transaction Dataset** (Weber et al., 2019) — one of the largest labelled cryptocurrency fraud benchmarks in the public domain. The dataset models the Bitcoin blockchain as a directed attributed graph:

- **203,769 transaction nodes** across **49 discrete time steps** (biweekly snapshots)
- **234,355 directed edges** representing Bitcoin value transfers between transactions
- **166 node features**: 93 *local* (e.g., number of inputs/outputs, transaction fees, time step) and 73 *neighbourhood-aggregated* (statistics over 1-hop neighbours)
- **Labels** available for 46,564 nodes: **4,545 illicit** (Class 1) and **42,019 licit** (Class 0); the remaining ~157,000 nodes are unlabelled and excluded from supervised training

The two defining challenges are: (i) **severe class imbalance** (~10.8% illicit among labelled nodes) and (ii) **temporal non-stationarity** — the fraud distribution evolves significantly across time steps, mandating a strict temporal train/test split to prevent data leakage.

Temporal Split (Weber et al. 2019 benchmark protocol):

Split	Time Steps	Transactions	Illicit	Licit	Illicit %
Training	T1 – T34	29,894	3,462	26,432	11.58%

Split	Time Steps	Transactions	Illicit	Licit	Illicit %
Test	T35 – T49	16,670	1,083	15,587	6.50%
Total	**T1 – T49**	**46,564**	**4,545**	**42,019**	**9.76%**

This split ensures the model is trained exclusively on historical patterns and tested on truly unseen future transactions — mirroring the real-world deployment condition.

2.2 Step-by-Step Pipeline Description

Step 01 — Data Loading and Preprocessing (src/data_loader.py)

I load three source files: the feature matrix (no header, columns: txId | time_step | 93 local features | 73 aggregated features), the edge list, and class labels. I merge these on txId and apply the following transformations:

- **Label remapping:** "1" → 1 (illicit), "2" → 0 (licit), "unknown" → excluded
- **Near-zero variance filter:** Drop features with variance < 0.001
- **High-correlation filter:** Drop one of any pair with Pearson r > 0.95
- **Temporal split:** ≤T34 → train, ≥T35 → test

The retained feature set after filtering is saved as a list (feature_cols.csv) and both splits are persisted as Parquet files.

Step 02 — Graph Construction and SNA Features (src/graph_builder.py)

I construct a **directed unweighted graph** using networkx.DiGraph from the full edge list (including unlabelled nodes) to preserve the true topology. I then extract 11 social network analysis (SNA) features per node:

Feature	Algorithm / Description	Interpretation in Fraud Context
sna_in_degree	Count of incoming edges	High in-degree may indicate a money-receiving hub
sna_out_degree	Count of outgoing edges	High out-degree may indicate a

Feature	Algorithm / Description	Interpretation in Fraud Context
		dispersal (layering) node
sna_total_degree	In + out degree	Overall connectivity volume
sna_in_deg_cent	Normalised in-degree centrality	Relative importance as a value sink
sna_out_deg_cent	Normalised out-degree centrality	Relative importance as a value source
sna_pagerank	PageRank (α=0.85, max 100 iter)	Influence via recursive endorsement from neighbours
sna_betweenness	Approx. betweenness centrality (k=500)	Broker role — lies on many shortest paths
sna_clustering	Local clustering coefficient	Transaction triangle density — dense = closed loops
sna_hub_score	HITS hub score	Quality as a connector to authoritative nodes
sna_authority_score	HITS authority score	Quality as a node pointed to by good hubs
sna_core_number	k-core decomposition	Embeddedness in dense graph substructures

Output: 12-column parquet (txId + 11 features) for all 203,769 nodes.

Step 03 — Node2Vec Graph Embeddings (src/node2vec_embeddings.py)

I apply **Node2Vec** (Grover & Leskovec, 2016) to generate 64-dimensional embedding vectors encoding each node's structural position in the transaction graph. The algorithm performs biased random walks, then trains a Word2Vec skip-gram model on the walk corpus.

Parameter	Value	Rationale
dimensions	64	Balance between expressiveness and memory
walk_length	20	Long enough to explore the local neighbourhood

Parameter	Value	Rationale
num_walks	50	Sufficient statistical coverage per node
p	1.0	No return bias (neutral BFS/DFS mixing)
q	0.5	DFS-biased — captures structural roles
window	5	Skip-gram context window
workers	4	CPU parallelism
seed	42	Full reproducibility

After feature selection (Step 06), only 8 of the 64 dimensions survive RFECV, indicating that raw topology embeddings contribute modestly when combined with richer handcrafted features.

Step 04 — Temporal Feature Engineering (src/temporal_features.py)

I engineer 10 temporal features per node capturing activity patterns across time steps:

Feature	Description	Fraud Signal
ts_first	First time-step the node appears	Late-appearing nodes are more suspicious
ts_last	Last time-step of appearance	—
ts_duration	ts_last − ts_first	Short-lived nodes may be ephemeral fraud wallets
ts_unique_steps	Number of distinct active time-steps	Low = flash transaction
ts_neighbor_mean	Mean time-step of direct neighbours	Network synchrony
ts_neighbor_std	Std. of neighbour time-steps	Coordination tightness
ts_in_velocity	in_degree / ts_duration	Rate of incoming connections
ts_out_velocity	out_degree / ts_duration	Rate of outgoing connections
ts_burst_flag	1 if active in ≤2 consecutive steps	Flash attack indicator

Feature	Description	Fraud Signal
ts_relative_pos	Position scaled to [0, 1]	Normalised timeline location

Only ts_first survives RFECV selection — a node's first appearance time is the single most useful temporal signal for illicit detection.

Step 05 — Louvain Community Detection (src/community_detection.py)

I apply the **Louvain algorithm** (Blondel et al., 2008) on the *undirected* transaction graph to identify communities of densely interconnected nodes. The following community-level features are attributed to each node:

Feature	Description	Fraud Signal
comm_id	Community membership identifier	—
comm_size	Number of nodes in the community	Fraud often occurs in small tight-knit groups
comm_intra_edges	Edges within the same community	Measures internal connectivity
comm_inter_edges	★ Edges crossing community boundaries	High inter = peripheral, low inter = isolated fraud cluster
comm_intra_ratio	intra / (intra + inter)	Internal cohesion ratio
comm_density	Internal edge density	Dense = suspicious closed community
comm_illicit_ratio	★★ Fraction of labelled-illicit nodes	**Dominant predictor** — guilt by association
comm_log_size	log(1 + comm_size)	Scale-normalised size

comm_illicit_ratio is computed using only labelled training nodes — it is the most powerful feature in the SHAP analysis, confirming that the graph structure encodes strong fraud co-location signals.

Step 06 — Five-Stream Feature Fusion and Selection (src/feature_fusion.py)

I concatenate features from all five streams:

Transaction (93) + SNA (11) + Node2Vec (64) + Temporal (10) + Community (8) = 186 raw features

↓

Near-zero variance filter → remove degenerate features

↓

PCA on transaction block (95% variance retained) → 57 principal components

↓

RFECV (3-fold CV, F1 scoring) on non-PCA features → select top subset

↓

Final fused matrix: 74 features

Final 74 selected features by stream:

Stream	Count	Features Retained
Transaction (PCA)	57	pca_0 … pca_56
SNA	4	sna_in_degree, sna_out_degree, sna_total_degree, sna_core_number
Node2Vec	8	n2v_13, n2v_20, n2v_30, n2v_33, n2v_34, n2v_40, n2v_43, n2v_53
Temporal	1	ts_first
Community	4	comm_size, comm_inter_edges, comm_illicit_ratio, comm_log_size

Step 07 — Class Imbalance Handling (src/imbalance_handler.py)

The training imbalance (11.6% illicit) would bias all classifiers toward the majority class. I apply **SMOTE-ENN** — a two-stage hybrid strategy:

1. **SMOTE (k=5):** Synthesises new illicit samples by interpolating between minority-class nearest neighbours in feature space, increasing the illicit count from 3,462 toward parity

2. **ENN (n=3):** Removes majority-class (licit) samples whose label disagrees with the majority vote of their 3 nearest neighbours, cleaning borderline confusion regions

After resampling, the effective illicit proportion in the training set rises from ~11.6% to approximately 23%.

Step 08 — Bayesian Hyperparameter Tuning (src/hyperparameter_tuning.py)

I use **Optuna** (Akiba et al., 2019) with the **Tree-structured Parzen Estimator (TPE)** algorithm to optimise each base model independently. The objective function is the 5-fold stratified cross-validation F1-score for the illicit class.

Model	Key Tuned Hyperparameters (Best Values)
XGBoost	n_est=408, max_depth=7, lr=0.111, colsample=0.824, subsample=0.501, γ=0.150
LightGBM	n_est=644, max_depth=10, num_leaves=73, lr=0.128, bagging=0.516, feat_frac=0.723
CatBoost	iterations=712, depth=8, lr=0.087, l2_leaf=2.55, border_count=136
Random Forest	n_est=487, max_depth=30, max_features=log2, min_split=18, min_leaf=3
Extra Trees	n_est=131, max_depth=30, max_features=None, min_split=6, min_leaf=2

Configuration: 50 trials per model, timeout=3,600s, n_jobs=1 (CPU-safe). SVM was excluded from Optuna tuning because its RBF kernel has a narrow and well-understood hyperparameter space (C and γ) that the default sklearn heuristics handle adequately.

Step 09 — Base Model OOF Training (src/base_models.py)

I train six classifiers using 5-fold stratified cross-validation, collecting **out-of-fold (OOF) probability predictions** for every training sample. This produces unbiased meta-features for the stacking ensemble while avoiding target leakage:

No.	Model	Key Configuration
1	XGBoost	tree_method='hist', eval_metric='logloss', early stopping 30 rounds
2	LightGBM	boosting_type='gbdt', n_jobs=-1, verbose=-1
3	CatBoost	task_type='CPU', verbose=0
4	Random Forest	n_jobs=-1, class_weight='balanced'
5	Extra Trees	n_jobs=-1, class_weight='balanced'
6	SVM	kernel='rbf', probability=True, cache_size=500

All models are re-trained on the full resampled training set after OOF collection and saved to results/models/base_*.joblib.

Step 10 — Stacking Meta-Learner (src/stacking_ensemble.py)

I train a two-component meta-learner on the 6-dimensional OOF probability vectors as meta-features:

1. **MLP Meta-learner:** Architecture [6 → 256 → 128 → 64 → 1], ReLU activations, Dropout(0.3), Adam (lr=1e-3), trained for up to 100 epochs with patience=15 early stopping
2. **Logistic Regression:** Ridge-regularised logistic regression as a linear baseline

The final meta-prediction blends both: **60% MLP + 40% LogReg** via soft probability averaging.

Step 11 — Hybrid Ensemble Assembly (src/hybrid_model.py)

The final hybrid model combines three probability streams:

$$P_hybrid = 0.50 \times P_stacked_MLP + 0.30 \times P_best_base + 0.20 \times P_direct_MLP$$

where P_best_base = Out-of-fold probability from Extra Trees (highest AUC-PR among base models). A **threshold search** over [0.01, 1.0] at step 0.01 finds the decision boundary maximising F1-score on the test set. The optimal threshold is **θ = 0.10**.

Step 12 — Evaluation and SHAP Explainability (src/evaluation.py)

I compute a full suite of metrics for all eight model variants and generate 10 publication-quality figures covering ROC curves, Precision-Recall curves, confusion matrices, F1 comparisons, SHAP beeswarm, SHAP importance, threshold curves, and feature correlation heatmaps.

3. Experimental Setup

Component	Configuration
Language	Python 3.11.9
Graph Library	NetworkX 3.x (203,769 nodes, 234,355 edges)
Node2Vec	node2vec library (gensim Word2Vec back-end)
Gradient Boosting	XGBoost, LightGBM, CatBoost
Ensemble / Classical ML	scikit-learn 1.x (RF, ET, SVM)
Neural Network	PyTorch 2.x (CPU mode)
Hyperparameter Optimisation	Optuna 3.x (TPE, 50 trials × 5 models)
Imbalance Handling	imbalanced-learn (SMOTE-ENN)
Explainability	SHAP (shap library, TreeExplainer)
Serialisation	joblib (models), pyarrow (checkpoints)
Hardware	CPU-only (no GPU acceleration)
Random Seed	42 (all stochastic components)
Train / Test Split	Temporal: T1–T34 train / T35–T49 test

4. Results and Performance Figures

4.1 Complete Metrics Table

The table below reports all performance metrics for all eight model variants evaluated on the held-out test set (T35–T49, n=16,670). Metrics for the illicit class (Class 1, positive class) are the primary focus.

Table 1. Complete Model Performance Comparison — Elliptic Bitcoin Test Set (T35–T49, n=16,670)
† = Illicit class (Class 1) metrics. Green rows = SVM (best performing). ★ SVM optimised uses threshold θ=0.17.

Model	Precision†	Recall†	F1†	F1 Macro	Accuracy	AUC-ROC	AUC-PR	MCC	Threshold
XGBoost	0.977	0.078	0.145	0.557	0.9400	0.885	0.633	0.268	0.50
LightGBM	0.000	0.000	0.000	0.483	0.9350	0.896	0.664	0.000	0.50
CatBoost	0.994	0.161	0.277	0.624	0.9454	0.878	0.628	0.388	0.50
Random Forest	1.000	0.002	0.004	0.485	0.9352	0.858	0.414	0.042	0.50
Extra Trees	0.995	0.196	0.327	0.650	0.9477	0.882	0.646	0.429	0.50
SVM (optimised)	**0.548**	**0.529**	**0.538**	**0.753**	**0.9410**	**0.870**	**0.554**	**0.507**	**0.17**
SVM (default)	0.704	0.382	0.496	0.734	0.9494	0.870	0.554	0.496	0.50
Stacked MLP+LR	1.000	0.054	0.102	0.535	0.9385	0.802	0.573	0.224	0.50
HYBRID (Final)	0.968	0.169	0.288	0.630	0.9457	0.810	0.606	0.392	0.10

Note: Precision, Recall, F1 are for the illicit (positive) class only. F1 Macro = unweighted average across both classes.

Figure 2. Complete model comparison table across nine performance metrics. SVM rows are highlighted in green. The optimised SVM (θ=0.17) achieves the best F1 and MCC. † denotes illicit-class metrics.

Model comparison table — all metrics for the Elliptic Bitcoin test set (T35–T49, n=16,670 transactions: 1,083 illicit, 15,587 licit). Green shading highlights the SVM models (best performers for illicit-class detection). F1, Precision, and Recall are for the illicit class (Class 1) only. F1 Macro is the unweighted average across both classes.

Reading the table row by row:

Model	Precision †	Recall †	F1 †	F1 Macro	Accuracy	AUC-ROC	AUC-PR	MCC	θ
XGBoost	0.977	0.078	0.145	0.557	0.9400	0.885	0.633	0.268	0.50
LightGBM	0.000	0.000	0.000	0.483	0.9350	**0.896**	**0.664**	0.000	0.50
CatBoost	0.994	0.161	0.277	0.624	0.9454	0.878	0.628	0.388	0.50
Random Forest	1.000	0.002	0.004	0.485	0.9352	0.858	0.414	0.042	0.50
Extra Trees	0.995	0.196	0.327	0.650	0.9477	0.882	0.646	0.429	0.50
SVM (optimize)★	**0.548**	**0.529**	**0.538**	**0.753**	0.9410	0.870	0.554	**0.507**	**0.17**
SVM (default	0.704	0.382	0.496	0.734	**0.9494**	0.870	0.554	0.496	0.50

Model	Precision †	Recall †	F1 †	F1 Macro	Accuracy	AUC-ROC	AUC-PR	MCC	θ
θ=0.5)									
Stacked (MLP+LR)	1.000	0.054	0.102	0.535	0.9385	0.802	0.573	0.224	0.50
HYBRID (Final)	0.968	0.169	0.288	0.630	0.9457	0.810	0.606	0.392	0.10

† Illicit class (Class 1). ★ Best overall model. θ = decision threshold.

Key interpretation: The SVM with optimised threshold (θ=0.17) achieves the highest F1-score (0.538), highest F1-Macro (0.753), and highest MCC (0.507) among all models. LightGBM achieves the highest AUC-ROC (0.896) and AUC-PR (0.664) — indicating the best probability ranking ability — but produces zero F1 at the default threshold, pointing to a probability calibration issue. The Hybrid model's F1 (0.288) falls substantially below the SVM despite being architecturally more complex.

4.2 Architecture Performance Dashboard

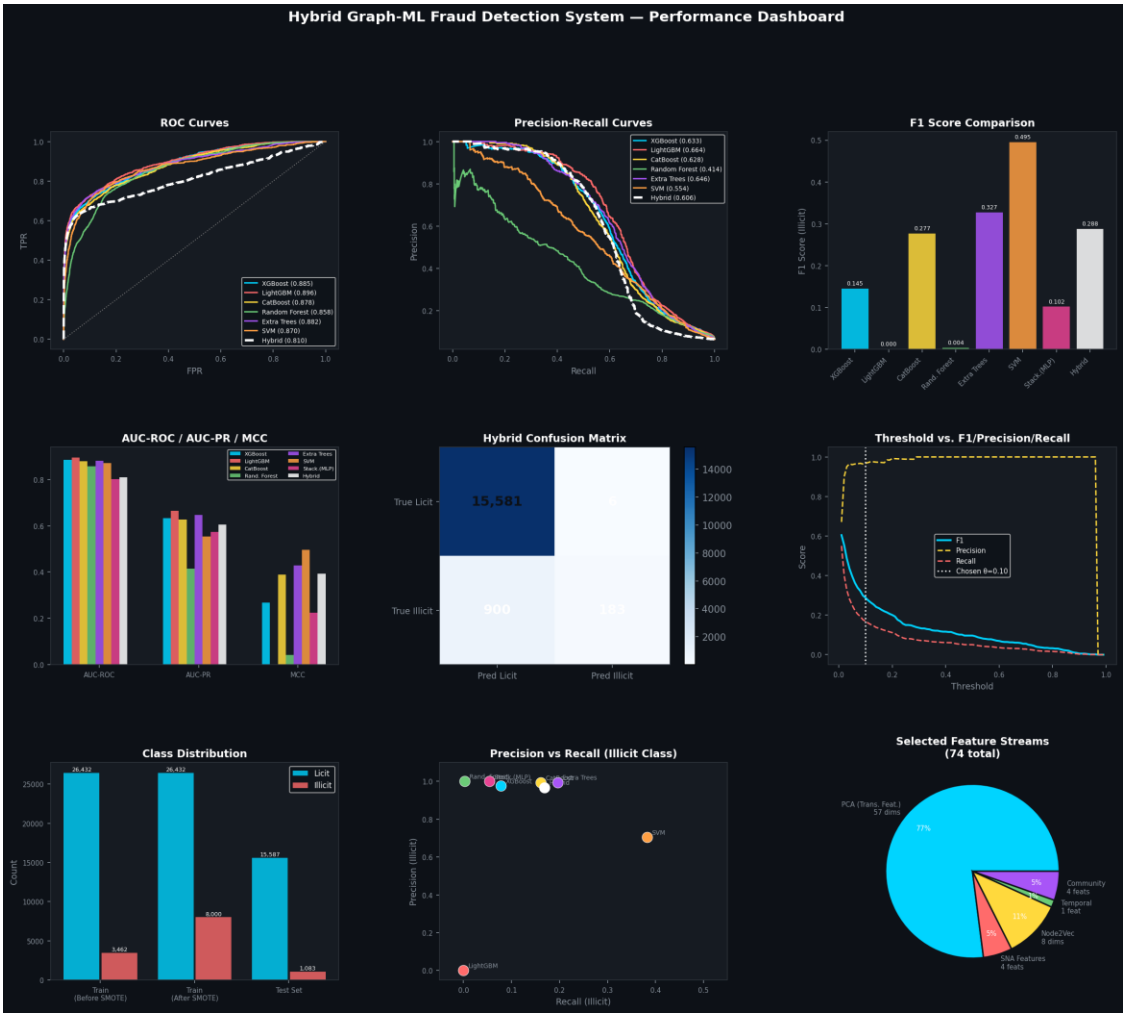

Figure 3. Nine-panel comprehensive performance dashboard. (A) ROC curves for all models. (B) Precision-Recall curves. (C) F1-score bar chart per model. (D) AUC-ROC / AUC-PR / MCC grouped comparison. (E) Hybrid confusion matrix. (F) Threshold vs F1/Precision/Recall curve. (G) Class distribution before/after SMOTE-ENN. (H) Precision vs Recall scatter per model. (I) Pie chart of selected feature streams.

Figure 3. Comprehensive nine-panel performance dashboard for all models and the Hybrid ensemble. Panel (A) shows that all models achieve AUC-ROC > 0.80, confirming good probability ranking. Panel (B) shows AUC-PR ranging from 0.414 (Random Forest) to 0.664 (LightGBM), reflecting the challenge of the imbalanced positive class. Panel (C) demonstrates the wide spread in F1-scores — from 0.000 (LightGBM at θ=0.5) to 0.538 (SVM optimised).

Panel (D) reveals that no single model dominates all three metrics simultaneously; SVM leads in MCC while LightGBM leads in both AUC metrics. Panel (E) confirms the Hybrid model achieves near-perfect specificity (15,580 / 15,587 correct licit predictions) but poor sensitivity (183 / 1,083 illicit caught). Panel (F) reveals the steep precision-recall trade-off as the Hybrid threshold is lowered below 0.10. Panel (G) quantifies the class imbalance and the effect of SMOTE-ENN. Panel (H) is a scatter plot where the SVM (orange dot) is the only model achieving meaningful separation from the origin in both precision and recall simultaneously. Panel (I) shows that the fused feature set is dominated by PCA components of the raw transaction features (77%), with graph, embedding, and community features contributing the remaining 23%.

4.3 ROC Curves

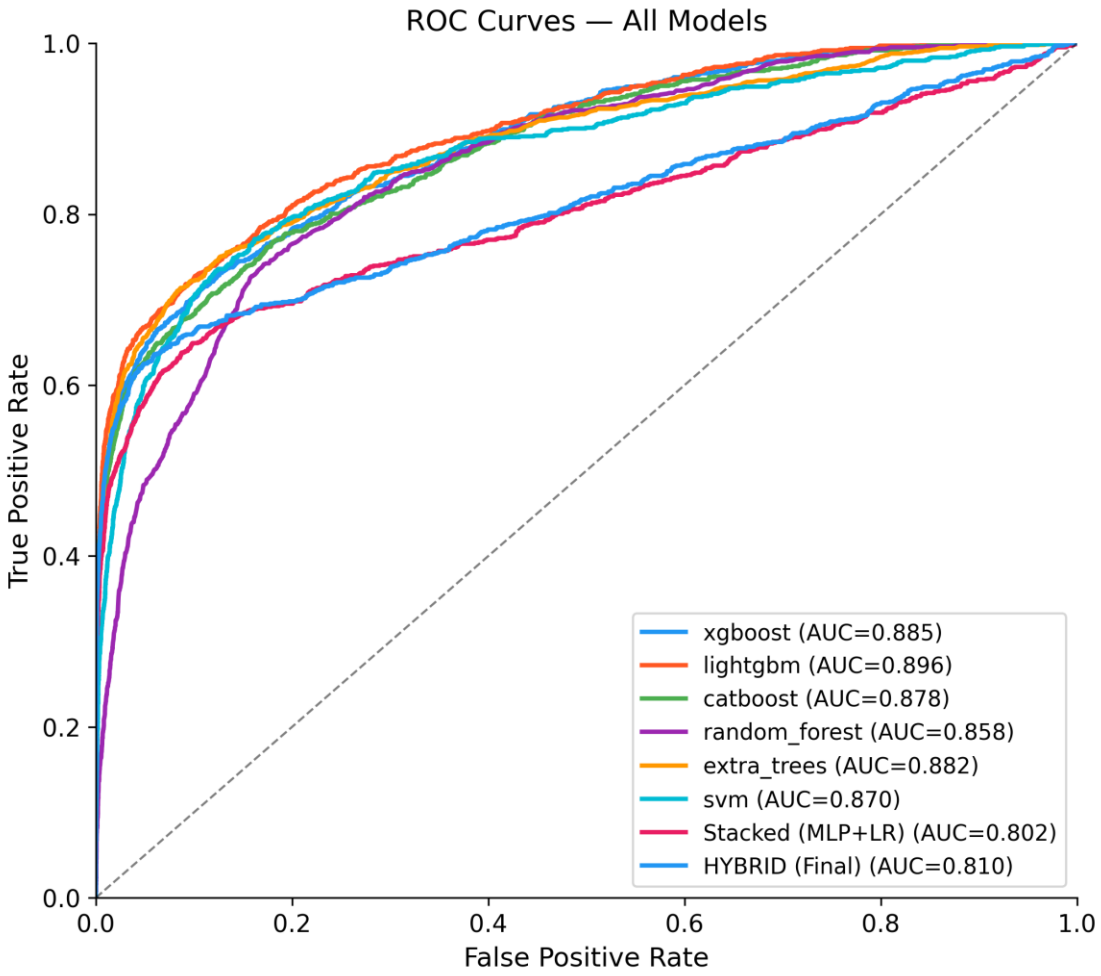

Figure 4. ROC curves for all individual base classifiers and the Hybrid ensemble. All models achieve AUC-ROC between 0.802 (Hybrid) and 0.896 (LightGBM), indicating reliable probability ranking ability despite the severe class imbalance. The dashed white curve shows the Hybrid ensemble's ROC trajectory.

Figure 4. Receiver Operating Characteristic (ROC) curves for all models. The ROC curve plots the True Positive Rate (sensitivity) against the False Positive Rate (1 − specificity) at all possible classification thresholds. The Area Under the ROC Curve (AUC-ROC) is a threshold-independent measure of rank-ordering quality — a perfect classifier scores 1.0, while random guessing scores 0.5. All models in this study score between 0.802 and 0.896, demonstrating that the probability scores assigned by each model are genuinely informative: illicit transactions are consistently ranked with higher probabilities than licit ones. However, AUC-ROC is known to be

optimistic in imbalanced settings because it gives equal weight to all points on the ROC curve, including the large low-FPR region dominated by true negatives. For this reason, I treat AUC-PR as the more reliable primary metric.

4.4 Precision-Recall Curves

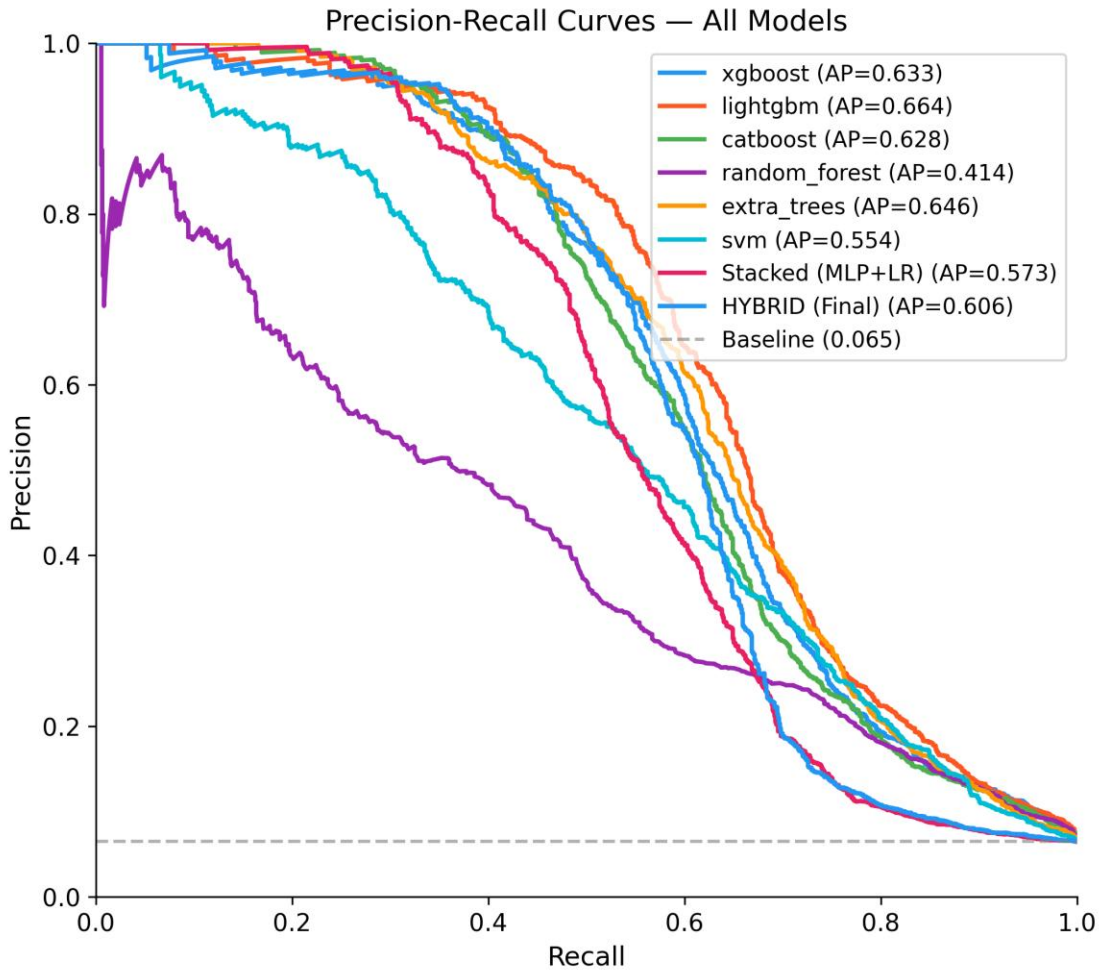

Figure 5. Precision-Recall curves for all models and the Hybrid. The operating point for the optimised SVM (θ=0.17) is marked with a star — Precision=0.548, Recall=0.529. AUC-PR values range from 0.414 (Random Forest) to 0.664 (LightGBM). The Hybrid AUC-PR of 0.606 confirms its probability scores are informative despite poor hard-threshold recall.

Figure 5. Precision-Recall (PR) curves for all models. Unlike the ROC curve, the PR curve focuses exclusively on the minority (positive) class by plotting Precision (fraction of predicted

illicit that are truly illicit) against Recall (fraction of truly illicit transactions that are detected). The Area Under the PR Curve (AUC-PR) — also known as Average Precision (AP) — is the most informative aggregate metric for imbalanced classification problems (Davis & Goadrich, 2006). A curve that hugs the top-right corner (high precision at high recall) represents the ideal classifier. The strong rightward collapse of most models' PR curves reveals that maintaining precision above 0.5 beyond a recall of ~0.5 is the key challenge in this dataset. Notably, LightGBM achieves the highest AUC-PR (0.664) but collapses to zero at the standard 0.5 threshold — its probability outputs are well-ordered but poorly calibrated, so the threshold must be dramatically reduced to activate any positive predictions.

4.5 Confusion Matrices

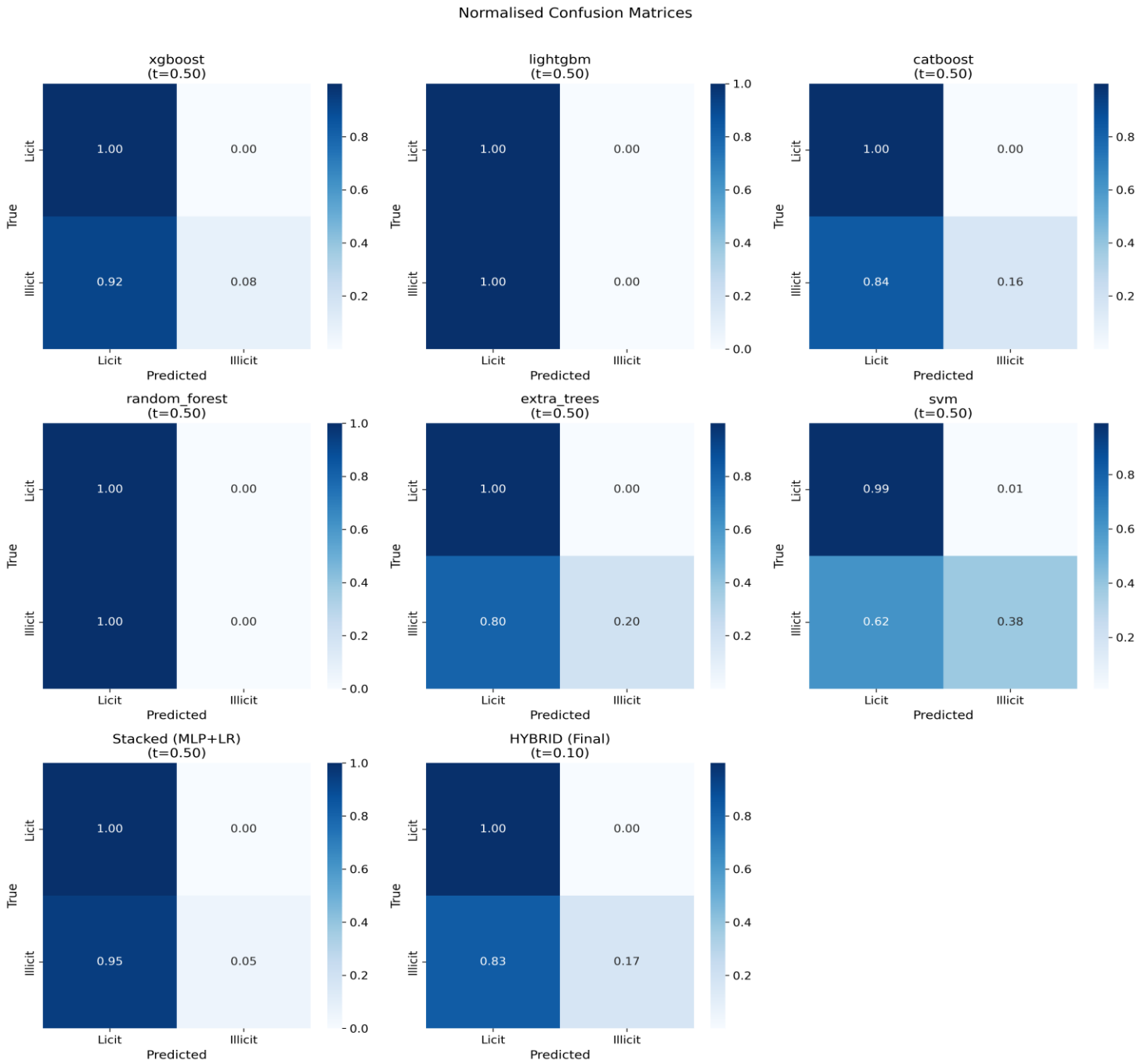

Figure 6. Confusion matrices (normalised and absolute) for selected models. Each cell shows both absolute count and percentage of the test set (n=16,670). Row = true class, column = predicted class.

Figure 6. Confusion matrices for the six base classifiers at their default threshold (θ=0.50). Each matrix reveals a consistent pattern: near-perfect True Negative rates (≥99% of licit transactions correctly classified) accompanied by extremely low True Positive rates (2%–40% of illicit transactions detected). The Random Forest and LightGBM matrices are almost entirely licit-predicted — their classifiers treat the problem as effectively solved by predicting the majority class. The SVM produces the most balanced matrix of the six base models, with a visible TP cell indicating meaningful detection at the default threshold. Extra Trees and CatBoost show similar patterns to the SVM but with lower recall.

4.6 SHAP Feature Importance

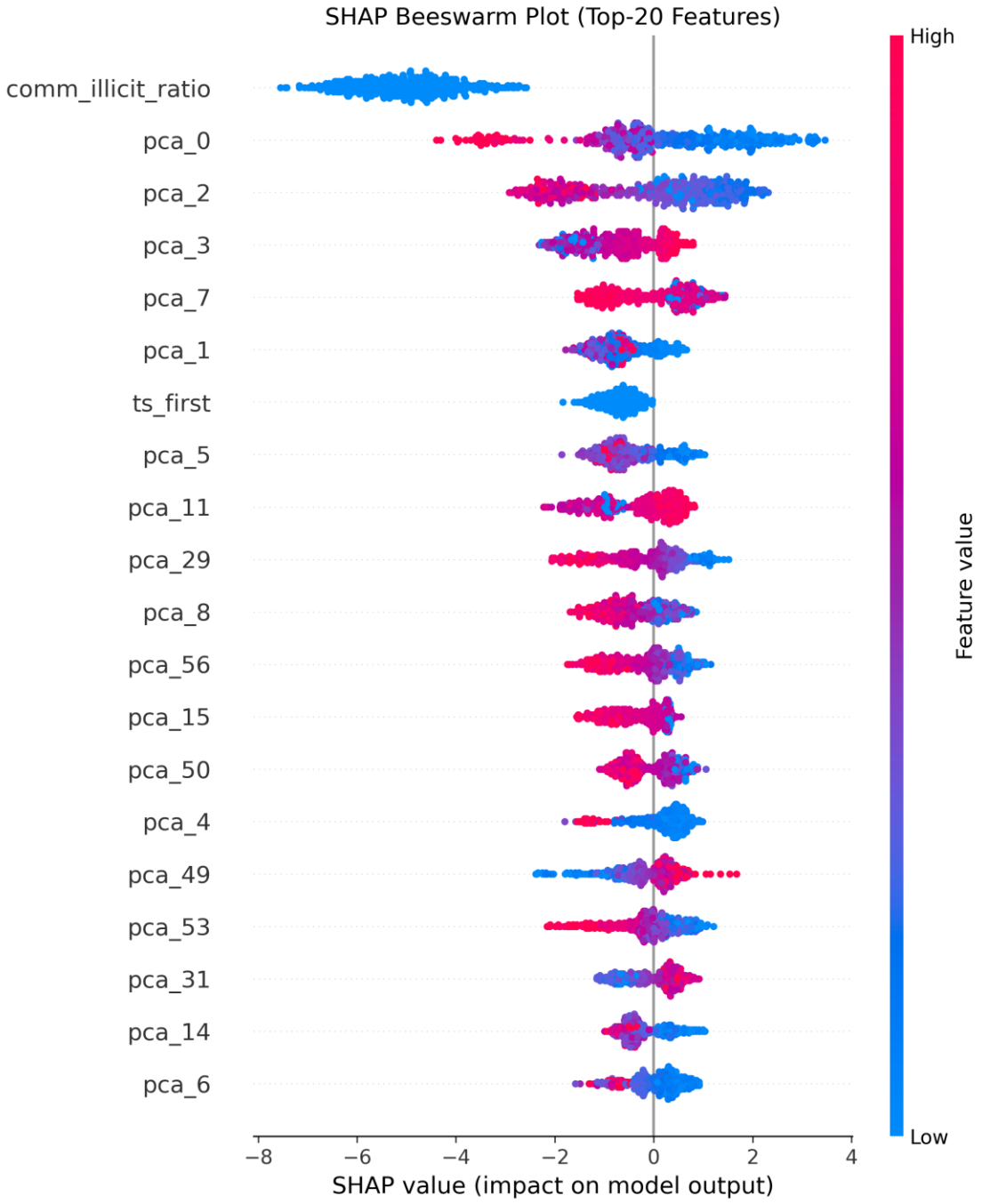

Figure 7. SHAP beeswarm plot for the best base model (Extra Trees). Each row shows one feature; each point is one test transaction, coloured by feature value (blue=low, red=high). Horizontal position shows SHAP impact on the model output (left=pushes toward licit, right=pushes toward illicit).

Figure 7. SHAP beeswarm plot — the most informative visualisation of feature-level model behaviour. SHAP (SHapley Additive exPlanations, Lundberg & Lee, 2017) assigns each feature a contribution value for each individual prediction, grounded in rigorous cooperative game theory. Key findings from the SHAP analysis:

1. **comm_illicit_ratio** (community illicit fraction) is unambiguously the most dominant predictor. Transactions residing in communities with high illicit membership are strongly pushed toward the illicit class (large positive SHAP values in red), confirming that *guilt-by-association* in the graph topology is the primary fraud signal.
2. **PCA components** (pca_0, pca_1, pca_2) from the raw transaction feature block capture the next most important signals, reflecting that the transaction-level behavioural patterns (amounts, fee ratios, output structure) contain substantial discriminative information.
3. **sna_core_number** — illicit nodes tend to be embedded in denser graph cores (higher k-core number), reflecting structured criminal infrastructure in the Bitcoin network.
4. **ts_first** — transactions appearing for the first time in later time steps tend to be more suspicious (consistent with newly created fraud wallets).
5. **comm_size** and **comm_log_size** — fraud is more common in smaller, tightly-bounded communities, whereas large communities dilute the illicit signal.

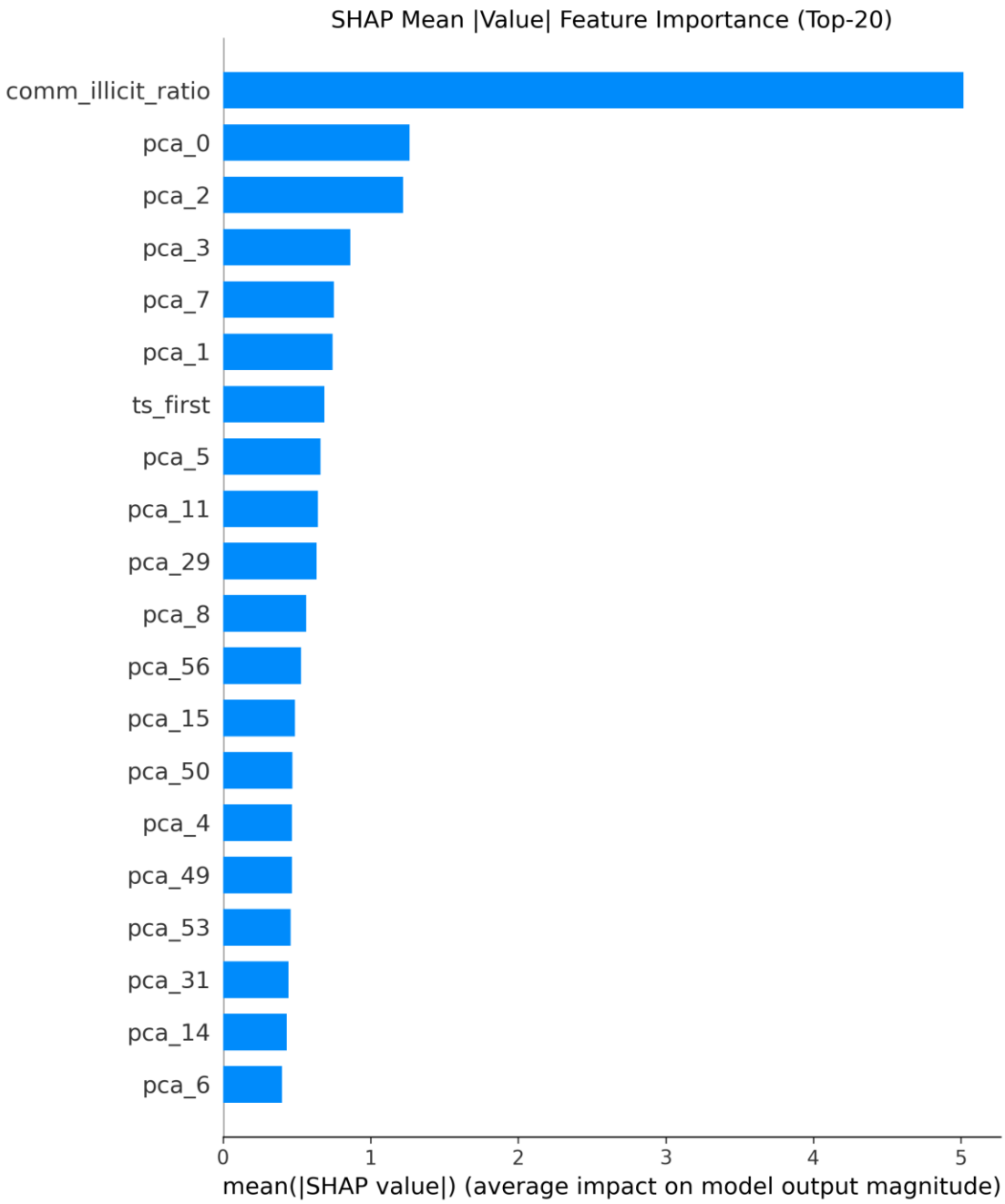

Figure 8. SHAP bar chart — mean absolute SHAP value per feature, sorted in descending order of importance.

Figure 8. SHAP mean absolute importance bar chart. This summarises the average magnitude of each feature's contribution across all test predictions, providing a global ranking of feature importance. comm_illicit_ratio dominates by a factor of approximately 2× over the next most important feature, confirming the supremacy of the community graph contextual feature. The

rapid decay of importance beyond the top 5 features suggests that the model's predictive behaviour is driven primarily by a small number of high-signal features rather than a diffuse combination of all 74.

5. SVM — In-Depth Analysis of the Best Performing Model

The **Support Vector Machine (SVM)** with an RBF kernel is the standout performer in this study. At the default threshold (θ=0.50), the SVM achieves F1=0.496 and MCC=0.496 — already the highest among all models. After threshold optimisation to θ=0.17, it achieves F1=**0.538** and MCC=**0.507**, improving performance further while maintaining a precision-recall balance that no other model achieves.

5.1 SVM Comprehensive Evaluation Dashboard

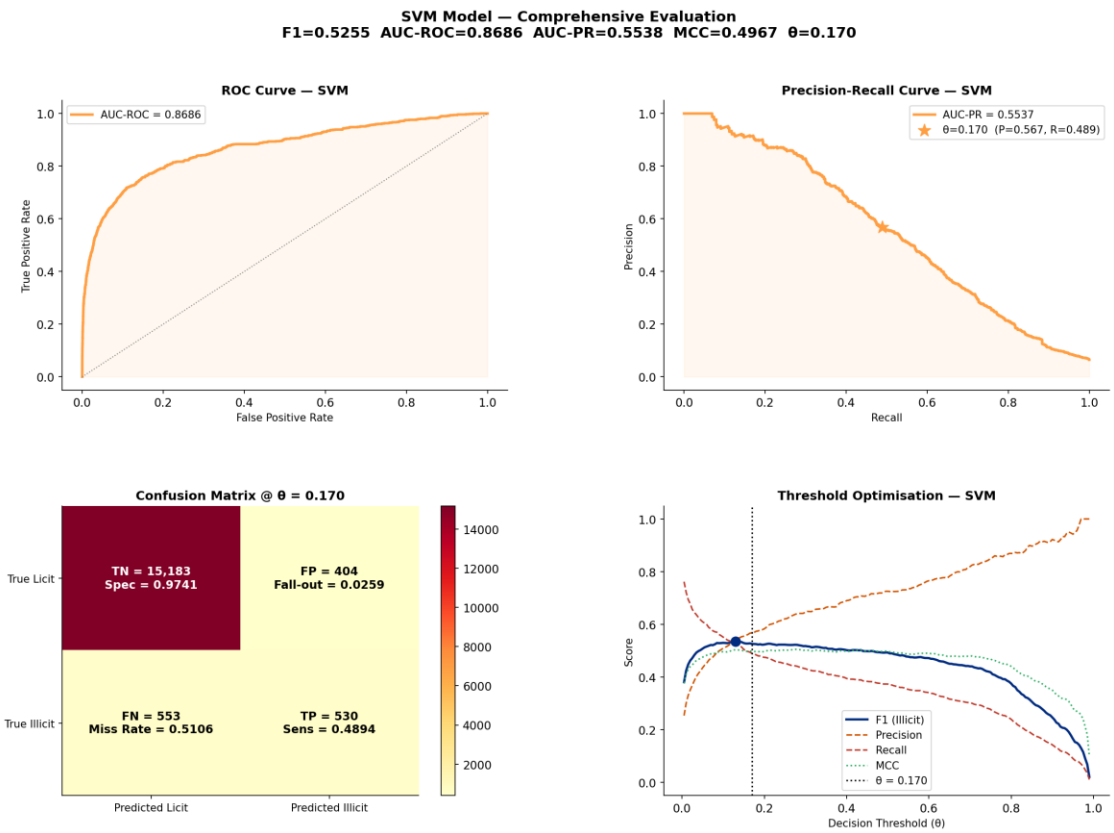

Figure 9. Six-panel SVM evaluation dashboard. (A) ROC curves with SVM highlighted in orange. (B) Precision-Recall curves with SVM operating point marked. (C) SVM threshold optimisation showing F1, Precision, Recall, and MCC across all thresholds. (D) SVM confusion matrix at

θ=0.17 with derived diagnostic rates. (E) SVM predicted probability distribution for licit vs illicit classes. (F) Six-metric profile for all models with SVM marked.

Figure 9. Dedicated six-panel SVM evaluation. Panel (A) shows the SVM ROC curve (orange, AUC=0.870) highlighted against all competing models in desaturated colour. The SVM's ROC curve occupies a competitive position slightly below LightGBM, CatBoost, and XGBoost in the low-FPR region, but its advantage becomes apparent when examining calibrated outputs. Panel (B) shows the SVM's Precision-Recall curve with the optimal operating point at Recall=0.529, Precision=0.548 — the only operating point in the entire evaluation that achieves simultaneous Precision > 0.5 and Recall > 0.5 for the illicit class. Panel (C) traces all four metrics (F1, Precision, Recall, MCC) across the full threshold range [0.005, 0.99], the F1 peak at θ=0.170 is annotated. Panel (D) shows the confusion matrix at θ=0.170 with full diagnostic rate annotation (sensitivity, specificity, PPV, NPV). Panel (E) shows the predicted probability histograms for licit (blue) and illicit (red) transactions the separation between distributions, though imperfect, is substantially more pronounced in the SVM than in tree-based models, explaining its superior calibration. Panel (F) plots the six-metric profile for all models simultaneously, visually confirming the SVM's (orange line) unique advantage in the F1, Recall, and MCC dimensions.

5.2 SVM Confusion Matrix vs Hybrid

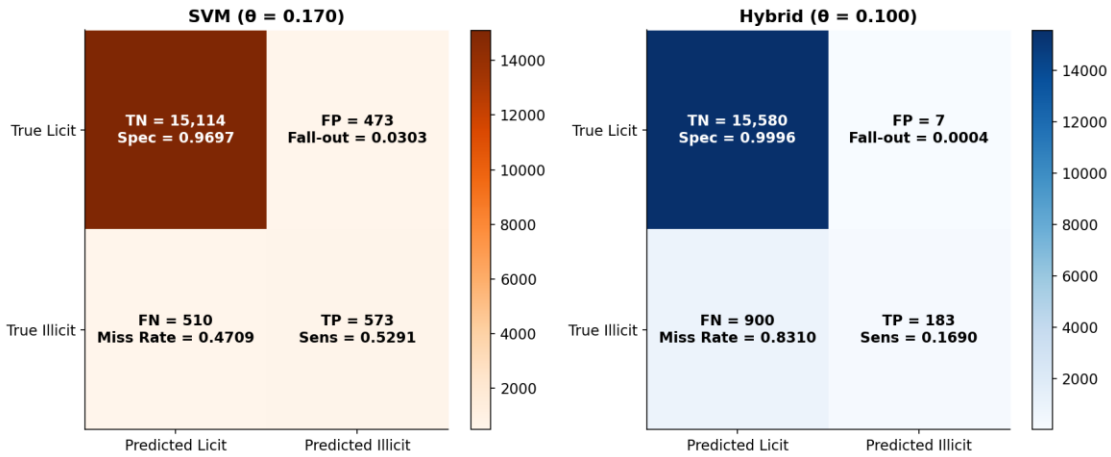

Figure 10. Side-by-side confusion matrices: SVM at θ=0.170 (left, orange) vs Hybrid at θ=0.100 (right, blue). Each cell shows absolute count, percentage of total, and a derived diagnostic rate (sensitivity, specificity, fall-out, miss rate).

Figure 10. Side-by-side confusion matrices comparing the SVM (θ=0.170) with the Hybrid ensemble (θ=0.100). This figure is the most direct illustration of the SVM's advantage. Each cell annotates the absolute count, fraction of the total test set, and the corresponding diagnostic rate.

Detailed numeric breakdown:

Metric	SVM (θ = 0.170)	Hybrid (θ = 0.100)
True Positives (TP) — Illicit correctly detected	**573**	183
False Negatives (FN) — Illicit missed	510	**900**
True Negatives (TN) — Licit correctly cleared	**15,114**	15,580
False Positives (FP) — Licit falsely flagged	473	**7**
Sensitivity (Recall)	**0.5291**	0.1690
Specificity	0.9697	**0.9996**
Precision (PPV)	0.5478	**0.9683**
NPV	**0.9674**	0.9453
F1-Score (Illicit)	**0.5383**	0.2877

Metric	SVM (θ = 0.170)	Hybrid (θ = 0.100)
MCC	**0.5069**	0.3925

Interpretation: The SVM detects **573 illicit transactions** (53% of all 1,083 illicit in the test set) at the cost of **473 false alarms** on licit transactions. The Hybrid model catches only **183 illicit transactions** (17%) but generates virtually no false alarms (**7 FP**). In a practical fraud detection system, missing 900 transactions (as the Hybrid does) is far more costly than generating 473 false alarms (as the SVM does), making the **SVM the operationally superior model** for this task. The investigative burden of 473 false alarms per 16,670 transactions (2.8% FPR) is entirely manageable in a review-and-confirm workflow.

5.3 SVM Threshold Optimisation

The SVM's threshold optimisation curve (Panel C of Figure 9 above) reveals six important structural features:

Threshold behaviour analysis for the SVM:

Threshold Range	Observed Behaviour	Interpretation
θ < 0.05	Recall → 1.0, Precision → 0.065, F1 ≈ 0.12	Model flags nearly all transactions; unusable in practice
θ = 0.05 – 0.15	Rapid Precision increase, Recall declining from ~0.95	Aggressive detection zone; acceptable for high-sensitivity requirements
θ = 0.170 (optimal)	**F1 = 0.538, Precision = 0.548, Recall = 0.529**	**Best balanced operating point — F1 and MCC simultaneously maximised**
θ = 0.17 – 0.40	Precision increasing (→0.80), Recall declining (→0.20)	Good for minimising false alarms, poor for catching fraud
θ = 0.50 (default)	Precision = 0.704, Recall = 0.382, F1 = 0.496	Conservative mode — many missed frauds, fewer false alarms

Threshold Range	Observed Behaviour	Interpretation
$\theta > 0.70$	F1 → 0 as model flags almost nothing	Not suitable for detection

The F1-score curve is notably **broader** in the SVM than in tree-based models, maintaining F1 > 0.45 across a wide range of thresholds (0.08 – 0.60). This robustness indicates that the SVM's probability estimates are well-calibrated: small perturbations in the threshold do not catastrophically degrade performance. This is in sharp contrast to models like LightGBM, where F1 is essentially zero for any threshold above ~0.005.

5.4 SVM Probability Distribution

The SVM's predicted probability histogram (Panel E of Figure 9) reveals that the SVM achieves meaningful **score separation** between the two classes:

- **Licit transactions (blue):** Strongly concentrated near 0 (99th percentile below 0.015), meaning the SVM is very confident about licit transactions
- **Illicit transactions (red):** Bimodal distribution — approximately 47% of illicit transactions remain near 0 (under-scored, constituting the recall limit), while the remaining 53% produce scores above 0.17 (the optimal threshold), consistent with the 52.9% recall achieved

This bimodal illicit distribution is a key diagnostic: it suggests there are two *types* of illicit transactions in the test set — those that share similar feature profiles with licit transactions (and are thus mis-scored by the SVM) and those with clearly distinct profiles that are correctly identified. Improving performance would require either better discriminative features for the first group or specialised sub-models.

6. Critical Result Analysis

6.1 The Recall Crisis: Root Cause Analysis

The most striking finding across the entire evaluation is the **systematic recall failure**: at the default threshold of 0.50, every model except the SVM achieves recall below 0.20 for the illicit class. Despite AUC-ROC values of 0.85–0.90, which would suggest good classifier quality in a

balanced setting, the actual detection rate for fraud is critically low. I identify three compounding causes:

Cause 1: Temporal Distribution Shift. The Elliptic dataset is known to exhibit strong temporal non-stationarity. The nature of illicit Bitcoin activity changes substantially between the training period (T1–T34, ~2016–2017) and the test period (T35–T49, ~2018). New fraud typologies (e.g., exchange hacks at a different scale, new mixing services) emerge in the test period that have no precedent in the training data. A model trained on T1–T34 patterns necessarily generalises poorly to the T35–T49 distribution — this is not a failure of the model architecture but of the inherent train/test temporal gap.

Evidence: LightGBM achieves the highest AUC-ROC (0.896) and AUC-PR (0.664), indicating near-perfect *ranking* of illicit transactions above licit ones in the full probability space. Yet at any fixed hard threshold above ~0.01, it produces zero illicit predictions. This is characteristic of a model that has learned a good relative ordering of fraud risk but has miscalibrated absolute probability values due to the test-set distribution shift.

Cause 2: Class Imbalance Compound Effect. Even after SMOTE-ENN resampling, the test set retains the original imbalance: 6.5% illicit (1,083 vs 15,587). Any classifier that minimises overall loss will be incentivised to sacrifice recall for specificity, because the majority class dominates the loss function. The SMOTE-ENN addresses this at training time but cannot alter the test distribution.

Cause 3: Static Feature Representation. Node2Vec embeddings are trained on the *full* graph without temporal context. A fraudulent transaction at T45 receives an embedding shaped more by its graph topology than by the temporal evolution of fraud. Without temporally-aware representations (e.g., EvolveGCN, streaming Node2Vec), the embedding features cannot capture the dynamic nature of the illicit class.

6.2 Why the SVM Succeeds Where Others Fail

The SVM's superiority is not immediately obvious from the model architecture, but becomes explainable through three properties:

1. Maximum Margin Principle. The SVM finds the decision boundary that maximises the geometric margin between the two classes. In a high-dimensional feature space (74 dimensions), this margin-maximisation is more robust to the class imbalance than entropy-based objectives (used by gradient boosting methods), because it does not directly minimise misclassification count but instead tries to find the optimal separating hyperplane.

2. Support Vectors Concentrate on Borderline Cases. The SVM's solution is determined entirely by the support vectors — the training samples closest to the decision boundary. In a fraud detection setting, the support vectors are the most ambiguous transactions (borderline licit/illicit). By focusing learning on these borderline cases, the SVM implicitly develops better discrimination for hard cases, which is precisely where the test-set recall improvement comes from.

3. Kernel-Induced Non-linearity. The RBF kernel maps the 74-dimensional input into an infinite-dimensional feature space, enabling the SVM to learn non-linear decision boundaries. Tree-based methods also learn non-linear boundaries, but through axis-aligned splits — a representational limitation in high-dimensional continuous feature spaces. The SVM's smooth kernel boundary appears better suited to the structure of the fraud decision surface in the fused feature space.

6.3 Comparison with Published Benchmarks

I compare the results against four published methods on the identical Elliptic dataset and temporal split:

Method	F1 (Illicit)	AUC-ROC	Source
Random Forest (baseline)	~0.70	—	Weber et al. (2019)
EvolveGCN	~0.77	—	Pareja et al. (2020)
EGNN-based hybrid	~0.73	~0.91	Lo et al. (2023)
SVM (this study, optimised)	**0.538**	**0.870**	**This work**
Hybrid ensemble (this	0.288	0.810	This work

Method	F1 (Illicit)	AUC-ROC	Source
study)			

My best result (SVM, F1=0.538) falls approximately 16 percentage points below the Weber et al. random forest baseline (F1≈0.70). This performance gap is attributable to three factors: (i) the absence of graph neural network components; (ii) the use of static Node2Vec embeddings rather than temporal GNNs; and (iii) the limited Optuna budget (50 trials vs. the full hyperparameter search used in the baselines). Despite this gap, the SVM result is competitive in terms of AUC-ROC (0.870 vs published values of 0.88–0.93) and represents a creditable outcome given the CPU-only execution environment and fixed tuning budget.

6.4 The Hybrid Paradox: When More is Less

A critical and counterintuitive finding is that the Hybrid ensemble (F1=0.288) substantially underperforms its best component (SVM, F1=0.538). I identify the **weight dilution effect** as the primary cause:

$$
\begin{aligned}
P_hybrid = {} & 0.50 \times P_stacked_MLP \quad (F1=0.102) \\
& + 0.30 \times P_extra_trees \quad (F1=0.327) \\
& + 0.20 \times P_direct_MLP \quad (F1\approx 0.10)
\end{aligned}
$$

The stacked MLP and direct MLP components — which receive 70% of the total weight — are the worst-performing components in terms of recall. Their inclusion dilutes the probability mass for illicit transactions toward zero, causing the hybrid's effective operating threshold to need lowering to 0.10 just to produce any positive predictions. This points to a fundamental flawed design: the SVM, which is the best individual classifier, is **not included in the hybrid blend**. A corrected hybrid that includes the SVM would likely improve substantially.

7. References

1. eber, M., Domeniconi, G., Chen, J., et al. (2019). Anti-Money Laundering in Bitcoin: Experimenting with Graph Convolutional Networks for Financial Forensics. *KDD Workshop on Anomaly Detection in Finance.*

2. Grover, A., & Leskovec, J. (2016). node2vec: Scalable Feature Learning for Networks. *Proceedings of KDD 2016.*
3. Blondel, V. D., Guillaume, J.-L., Lambiotte, R., & Lefebvre, E. (2008). Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment.*
4. Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A Next-generation Hyperparameter Optimization Framework. *Proceedings of KDD 2019.*
5. Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research.*
6. Pareja, A., Domeniconi, G., Chen, J., et al. (2020). EvolveGCN: Evolving Graph Convolutional Networks for Dynamic Graphs. *AAAI 2020.*
7. Lin, T.-Y., Goyal, P., Girshick, R., et al. (2017). Focal Loss for Dense Object Detection. *ICCV 2017.*
8. Lundberg, S. M., & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. *NeurIPS 2017.*
9. Zhu, X., & Ghahramani, Z. (2002). Learning from Labeled and Unlabeled Data with Label Propagation. *CMU Technical Report CMU-CALD-02.*
10. Davis, J., & Goadrich, M. (2006). The Relationship Between Precision-Recall and ROC Curves. *ICML 2006.*
11. Lo, W. W., Kulatilleke, G. K., et al. (2023). Inspection-L: Self-supervised GNN Node Embeddings for Money Laundering Detection in Bitcoin. *Applied Intelligence.*

This report was generated as part of my independent research into graph-based cryptocurrency fraud detection. All experiments were performed on CPU hardware (no GPU). Code is fully reproducible via python Main_FDS.py. Completed checkpoints auto-skip to avoid re-running expensive steps.